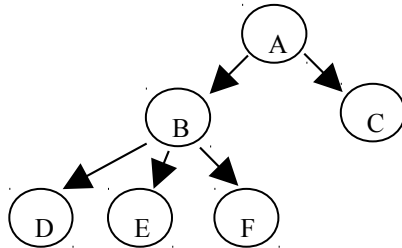


Árvores: conceitos gerais

Conceito: conjunto de elementos (nodos ou nós) entre os quais é mantida uma relação de hierarquia ou de composição.



Exemplo: o nodo A tem dois filhos, B e C. O nodo B tem 3 filhos, D, E e F.

Raiz: nodo que dá origem à árvore, não tem antecessor.

Cada nodo da árvore é raiz de uma subárvore, que por sua vez também é uma árvore.

Cada nodo pode ter no máximo um antecessor direto, e qualquer número de sucessores.

Nodo filho: nodo sucessor direto de um outro nodo.

Nodo pai: nodo antecessor direto de um outro nodo.

Nodos irmãos: nodos que têm o mesmo antecessor direto (o mesmo pai).

Nodo terminal ou **folha:** nodo que não possui filhos.

Nodo não terminal ou **interno:** nodo que possui filhos (no mínimo um filho).

Grau de um nodo: número de subárvores de um nodo.

Caminho entre n_1 e n_k : sequência de nodos de n_1 até n_k , na qual n_2 é filho de n_1 , ..., n_k é filho de n_{k-1} .

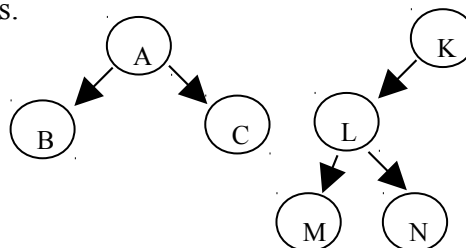
Comprimento de um caminho: número de nodos de um caminho menos um.

Profundidade de um nodo: comprimento do caminho da raiz da árvore até o nodo.

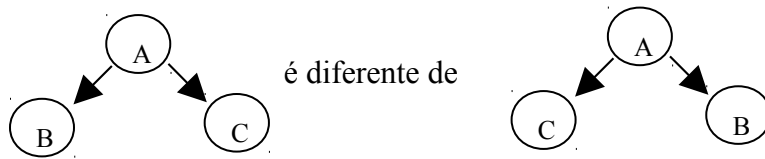
Todos os nodos de profundidade k estão no **nível** k da árvore.

Floresta: conjunto de árvores disjuntas.

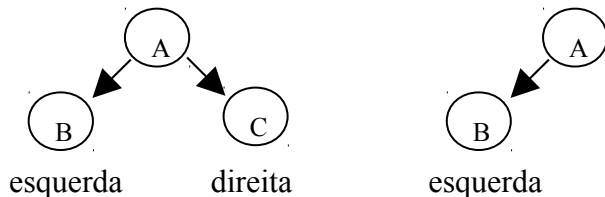
Exemplo: floresta de duas árvores, com raízes A e K. A tem 2 filhos, B e C. K tem um filho, L, e L tem dois filhos, M e N.



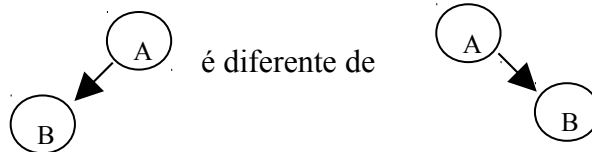
Árvore ordenada: a ordem das subárvores é significativa. Por exemplo, A com filhos B (à esquerda) e C (à direita) é diferente de A com filhos C (à esquerda) e B (à direita):



Árvore binária: grau de cada nodo varia de 0 a 2: um nodo pode não ter filhos, pode ter apenas uma subárvore à esquerda, pode ter apenas uma subárvore à direita, ou pode ter duas subárvores, uma à esquerda e outra à direita. Por exemplo: a árvore de raiz A pode ter dois filhos, B à esquerda e C à direita. Ou A pode ter apenas um filho, B, à esquerda:



Por definição, a árvore binária é ordenada. Então se a árvore de raiz A tiver o nodo B como filho à esquerda, não é a mesma árvore de raiz A que tem B como filho à direita:



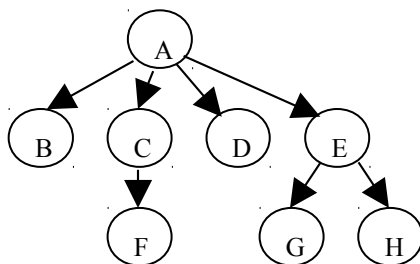
Operações sobre árvores:

```

TAD árvore{
  Dados: raiz
  Operações:
    consultaPai(E: nodo; S: nodo)
    consultaFilhos(E:nodo; S: lista de nodos)
    consultaIrmaos(E: nodo; S: lista de nodos)
    consultaNumeroNodos(E:raiz; S: valor)
    insere(E: nodo, nodopai)
    insereComIrmao(E: nodo, nodoirmao)
    remove(E: nodo)
    consultaEhFolha(E: nodo; S: info)
    ...
  }
  
```

Representação/implementação de árvores:

Considere uma árvore com a raiz A, sendo que A tem quatro filhos: B, C, D e E. O nodo C tem um filho: F. E o nodo E tem dois filhos: G e H.



Essa árvore pode ser representada em um programa de diferentes maneiras, utilizando vetores, listas encadeadas ou combinações dessas estruturas. A escolha da representação deve levar em conta principalmente quais operações serão executadas sobre a estrutura, e deve ser escolhida uma representação que facilite a execução das operações.

A seguir temos exemplos de representações por array (vetores) e por listas encadeadas, além de combinações dessas estruturas.

- Array (representação sequencial):
um vetor, no qual cada dupla de posições permite representar um nodo e seu número de filhos. Então A seguido de 4 significa que A tem 4 filhos, e assim por diante.

A	4	B	0	C	1	F	0	D	0	E	2	G	0	H	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Vantagens: interessante para armazenamento em meio permanente, arquivo (não para manipulação em geral)

Desvantagens: inserções, remoções, localização dos nodos (achar o pai, ...)

- Array (representação por pai):
um vetor, no qual cada posição terá o índice do nodo pai

A B C D E F G H

1 2 3 4 5 6 7 8

0	1	1	1	1	3	5	5
---	---	---	---	---	---	---	---

- índices, correspondem aos nodos (1=A, 2=B, 3=C, ...)

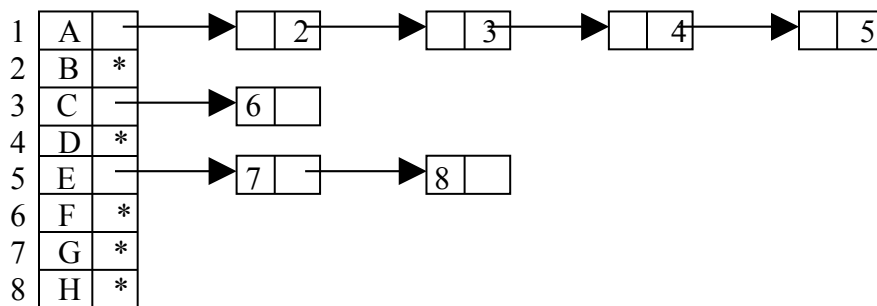
- o conteúdo de cada posição é o índice do nodo pai.

Assim, a posição 2 que corresponde ao nodo B tem como conteúdo 1, que é o índice do nodo A, pai do B

Vantagens: interessante para saber o pai dos nodos

Desvantagens: não facilita obtenção de informações sobre filhos, além das dificuldades nas inserções e remoções

- Listas encadeadas (listas de filhos)
um vetor com uma posição para cada nodo. Cada posição do vetor tem também uma referência para a lista encadeada dos nodos-filhos. Assim, por exemplo, se o vetor contém os nodos A, B, C, D, E, F, G e H, a posição do nodo A também tem a referência para uma lista encadeada com as posições 2, 3, 4 e 5, que são as posições dos nodos B, C, D e E no vetor.

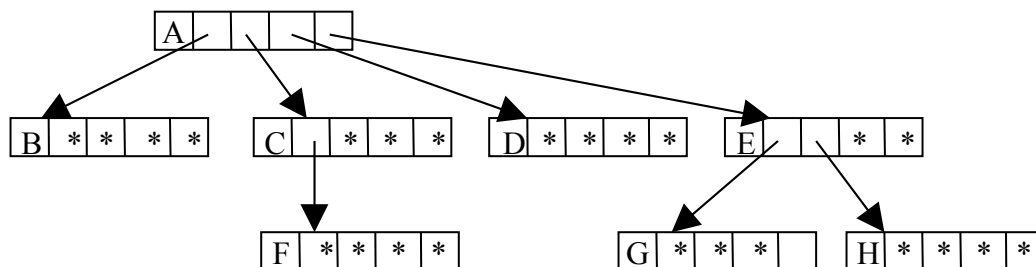


Cada nodo-filho das listas encadeadas tem que estar no vetor de nodos.

Vantagens: flexibilidade para manipulação de nodos filhos

Desvantagens: número máximo de nodos limitado

- Listas encadeadas (encadeamento de nodos)
cada nodo da árvore contém o valor do nodo e referências para os nodos filho, que também são representados dessa forma. Assim, cada nodo possui, além do campo (posição) para o valor do dado, campos de referências (ou de endereços) de acordo com o grau máximo da árvore. Se o grau máximo da árvore for 4, então cada nodo terá 4 campos para 4 referências a outros nodos.



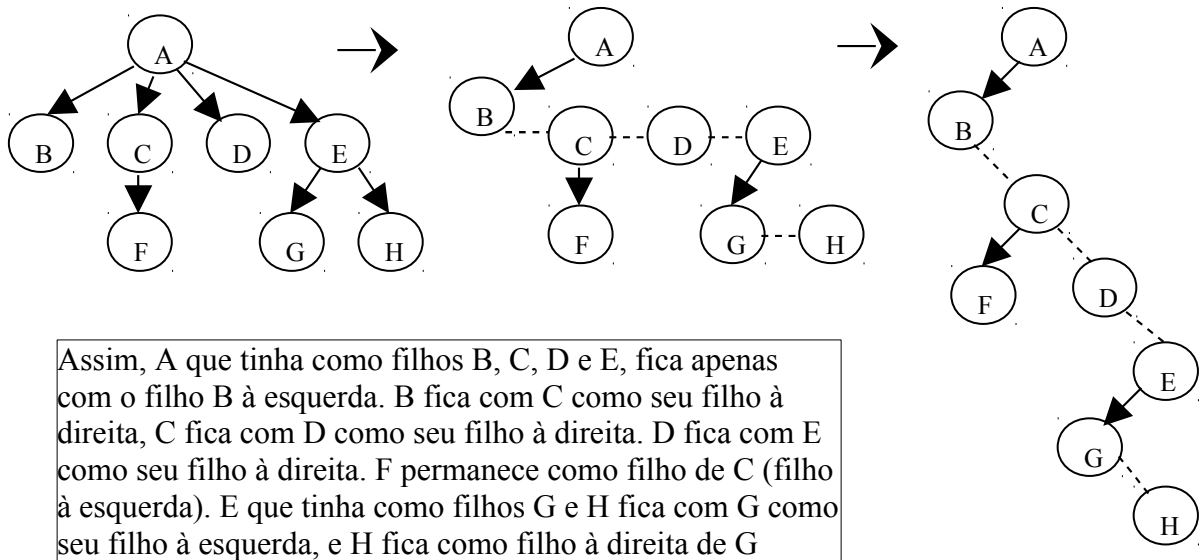
Vantagens: maior flexibilidade e eficiência na manipulação

Desvantagens: Muitos campos de endereço podem ficar vazios (desperdício de espaço). Nem sempre é possível prever o grau máximo da árvore.

Uma alternativa para tornar mais econômica a representação, é transformar a árvore n-ária em binária. Desta forma, todos os nodos terão uma representação com dois campos ponteiros, independente do grau máximo da árvore. A seguir, um algoritmo que permite transformar uma árvore com qualquer número de filhos em árvore binária.

Transformação de uma árvore qualquer em árvore binária

- ligar os nodos irmãos;
- remover as ligações entre pais e filhos, com exceção da ligação do nodo com seu primeiro filho



Interpretação da árvore binária:

- subárvore da esquerda de um nodo = filho real do nodo
- subárvore da direita de um nodo = irmão do nodo

A partir da árvore binária gerada, é possível transformá-la novamente na sua versão original, tornando os filhos à direita irmãos, e mantendo a ligação do filho à esquerda como filho original.